Routing in ASP.NET Core Web API

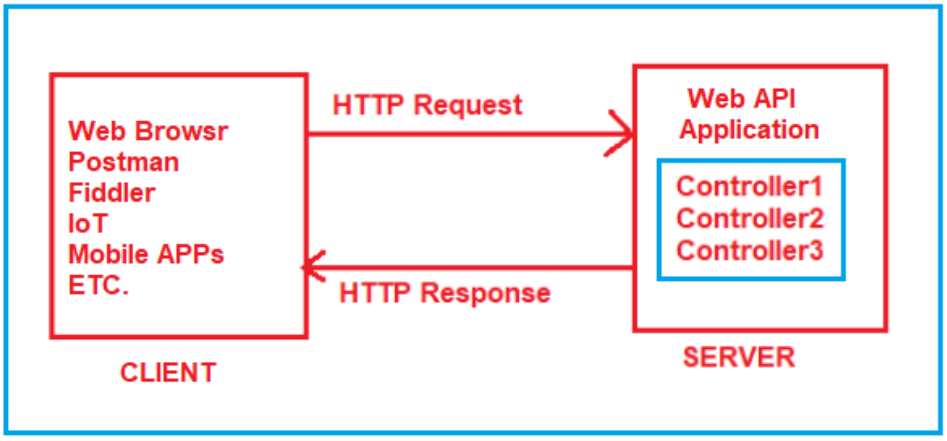
Back to: ASP.NET Core Web API Tutorials

Routing in ASP.NET Core Web API Application

In this article, I will discuss **Routing in ASP.NET Core Web API** Applications. Please read our previous article discussing **Models in ASP.NET Core Web API** Applications.

Why Routing in ASP.NET Core Web API?

In an ASP.NET Core Web API application, **Routing** is a fundamental concept that connects the **incoming HTTP request** from a client to the correct **controller action method** within the server application. Let us understand routing in ASP.NET Core Web API with an example. Please have a look at the following image.



On the left-hand side, we have multiple **clients**, such as:

- A **Web Browser** (e.g., Chrome, Edge)
- **Postman** or **Fiddler** (API testing tools)
- **Swagger UI** (auto-generated API explorer)
- A **Mobile App** or a **Desktop Application** consuming your API

On the right-hand side, we have the **server** hosting our **ASP.NET Core Web API application**. Inside this application, there are multiple **controllers**, and each controller contains one or more **action methods**, which define how specific HTTP requests should be handled.

Suppose the client is sending a request to the server. As we already discussed, the request must go through the Request Processing Pipeline. If everything is fine, then the ASP.NET Core Framework navigates that request to the controller action method. The controller action method handles the request, generates the response, and sends it back to the client who initially made the request.

Here, we need to understand how the framework will determine which request will be mapped to which controller action method. Basically, the mapping between the URL and the Resource (controller action method) is nothing but the concept of Routing.

What Is Routing in ASP.NET Core and How Does It Work?

In ASP.NET Core Web API, **Routing** is the process that maps incoming HTTP requests to the corresponding **Controller Actions**. Think of it as a **Traffic Director**. It examines the URL and decides which controller and action method should handle the request.

In earlier versions of ASP.NET Core, developers had to call two middleware components: explicitly

- **UseRouting()** → to match the request to an endpoint
- **UseEndpoints()** → to execute the matched endpoint

However, starting from **.NET 6 and later**, these are automatically configured when we use endpoint mapping methods such as: **app.MapControllers();**

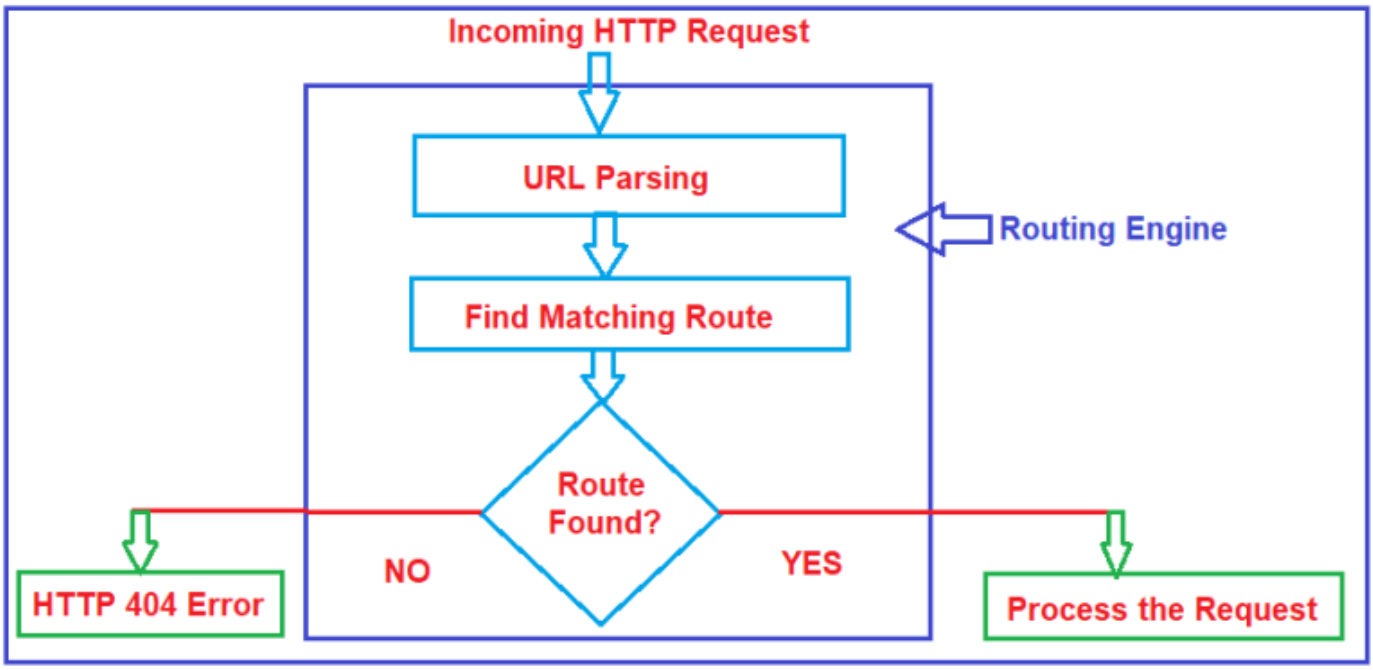
So, when we call the **app.MapControllers()**, the framework internally:

1. Adds the **Routing Middleware (**UseRouting()) to match incoming requests to controller actions.
2. Adds the **Endpoint Middleware** (UseEndpoints()) to invoke the matched controller action.

This means we no longer need to write UseRouting() and UseEndpoints() explicitly. They are automatically wired under the hood when using MapControllers().

How Routing Works in ASP.NET Core

To understand how Routing works in ASP.NET Core, please have a look at the following diagram. Here, in the image below, the Routing Engine represents both Routing Middleware and Endpoint Middleware**.**



Let us understand how Routing Works based on the above diagram:

Step 1: Incoming HTTP Request

When a client (such as a browser, Postman, Swagger UI, or mobile app) sends a request to the web server, it includes:

- **HTTP Method:** The type of action requested (GET, POST, PUT, DELETE, PATCH).
- **URL:** The target endpoint (e.g., https://example.com/api/customers/5).
- **Headers:** Metadata like Content-Type, Authorization, or Accept.
- **Query String (optional):** Additional parameters **sent as part of the URL** (e.g., ?search=abc).
- **Request Body (optional): Data sent with POST, PUT, or PATCH requests (e.g., JSON payload for creating or updating a resource)**.

Once received, the request flows through the ASP.NET Core **Middleware Pipeline**. One of the essential middleware components in this pipeline is the **Routing Middleware**, which starts the process of matching the request to an endpoint.

Step 2: URL Parsing (Handled by Routing Middleware)

When the Routing Middleware is invoked, it first parses the incoming URL into its components. For example, consider the request:

- **https://example.com/api/customers/5?search=abc**

This URL is broken down into:

- **Scheme: https** (communication protocol)
- **Host: example.com** (domain)
- **Path: /api/customers/5** (endpoint path)
- **Query String:** Additional parameters if provided (e.g., **?search=abc**)

The **Routing Middleware** uses this path and HTTP method to find a matching **endpoint** among those registered in the application during startup.

Step 3: Finding a Matching Endpoint

After the URL is parsed, the **Routing Middleware (UseRouting)** begins searching for a **matching endpoint** among all endpoints registered during application startup.

In **ASP.NET Core**, the traditional Route Table used in older ASP.NET MVC no longer exists. Instead, routing is based on a set of **Endpoint objects**, maintained inside one or more **Endpoint Data Sources**. Each **Endpoint** represents a concrete destination for a request and includes:

- A **Route Pattern** (for example, /api/customers/{id?})
- A **Request Delegate**, the code that will run (such as a controller action, Razor Page handler, or minimal API delegate)
- **Metadata** such as supported HTTP methods, authorization requirements, filters, and custom constraints

The **Routing Middleware** performs the following steps:

1. **Path Extraction:** It reads the request path (e.g., /api/customers/5) and HTTP method (e.g., GET).
2. **Pattern Matching:** It compares these against all configured endpoint patterns within the **Endpoint Data Sources**.
3. **Constraint Evaluation:** If multiple endpoints match, the routing engine applies additional constraints (like HTTP method or parameter types) to select the most suitable endpoint.
4. **Endpoint Assignment:** When a match is found, the selected **Endpoint** and its extracted **Route Values** (e.g., { “controller”: “Customers”, “action”: “GetById”, “id”: “5” }) are attached to the HttpContext.
5. **Prepare for Execution:** These details are stored in the context for use by the **Endpoint Middleware (UseEndpoints)**, which will later invoke the correct request delegate.

If no matching endpoint is found, the pipeline continues without an assigned endpoint, and ASP.NET Core will eventually return an **HTTP 404 Not Found** response.

Step 4: Endpoint Found or Not?

At this stage, the **Routing Middleware** has either identified a matching endpoint or found none. The outcome determines the next step in the request pipeline.

Scenario 1: No Matching Endpoint (HTTP 404 Error)

- If no endpoint matches the request, the HttpContext remains without an assigned endpoint.
- The request continues through the remaining middleware, but since **Endpoint Middleware** has nothing to execute, the framework finally returns an **HTTP 404 Not Found** response to the client.
- This indicates that the requested URL or resource does not exist in the application.

Scenario 2: Matching Endpoint Found (Process the Request)

- When a valid endpoint is found, the **Routing Middleware** attaches the endpoint and extracted **Route Values** to the HttpContext.
- Control is then passed to the **Endpoint Middleware (UseEndpoints)**, which invokes the endpoint's associated **Request Delegate**, typically a controller action, Razor Page handler, or minimal API method.
- From here, the framework performs **Model Binding**, executes **Filters** (like authorization or logging), and then runs the selected **Action Method** to process the request and generate a response.

Step 5: Policy Middlewares Use the Selected Endpoint (Optional)

Once the **Routing Middleware (UseRouting)** has successfully matched the incoming request to a specific **endpoint**, ASP.NET Core allows several **Policy Middlewares** to act **before the endpoint is executed**.

These middlewares enforce cross-cutting concerns such as **Authorization**, **CORS**, and **Rate Limiting**, using the metadata already attached to the selected endpoint. These are often referred to as **Policy Middlewares**, because they apply **policies** defined in the endpoint's metadata.

Common Policy Middlewares Include:

UseCors()

1. Applies Cross-Origin Resource Sharing rules.
2. Checks whether the current request's origin, headers, and method are allowed by the configured CORS policy.
3. Can be configured globally or per-endpoint using [EnableCors] or [DisableCors] attributes.

UseAuthentication()

- Authenticates the user based on tokens, cookies, or credentials.
- Populates the HttpContext.User property with the identity and claims if authentication succeeds.
- Runs before authorization, because authorization depends on the authenticated identity.

UseAuthorization()

- Uses endpoint metadata (like [Authorize] attributes or authorization policies) to decide whether the current request is allowed to proceed.
- If the user is not authorized, it short-circuits the pipeline with a **403 Forbidden** or **401 Unauthorized** response.
- If authorized, the request continues to the next middleware.

UseRateLimiter()

- Enforces rate-limiting policies attached to endpoints.
- Controls how many requests a client can make within a defined time window.

Step 6: Endpoint Execution (Endpoint Middleware Runs)

Once the Routing Middleware and optional Policy Middlewares have completed their tasks, the request finally reaches the **Endpoint Middleware**. This middleware is responsible for **executing the selected endpoint's request delegate**, which actually processes the request and generates the response.

When the application reaches the call to the app.UseEndpoints(…) or any of the Map*() methods (like MapControllers(), MapRazorPages(), or MapGet()), the **Endpoint Middleware** checks whether an endpoint has been selected by the Routing Middleware.

At this stage, the Endpoint Middleware performs the following steps:

1. Check for a Selected Endpoint

- The Endpoint Middleware retrieves the endpoint information from the current HttpContext using: **var endpoint = context.GetEndpoint();**
- If **no endpoint** was selected (i.e., routing failed to find a match), the middleware simply passes control to the next middleware in the pipeline. Eventually, the framework returns an **HTTP 404 Not Found** response.
- If an **endpoint exists**, the Endpoint Middleware **invokes the associated Request Delegate**, the exact code that should handle the request.

Each endpoint corresponds to a specific type of handler:

- For **MVC Controllers**, it invokes the **Action Invoker** that executes the controller's action method.
- For **Razor Pages**, it executes the corresponding **Page Handler**.
- For **Minimal APIs**, it directly invokes the **lambda delegate** defined in the MapGet, MapPost, or similar method.

2. Dependency Injection and Controller Creation

- When a controller-based endpoint is invoked, the **Dependency Injection (DI) container** creates a new instance of the controller.
- Any dependencies (such as repositories, services, or loggers) defined in the controller's constructor are automatically resolved from the DI container.
- For Minimal APIs, dependencies are injected directly into the handler's parameters.

3. Model Binding

- Once the controller or handler is ready, **Model Binding** takes place.
- The framework extracts the incoming data from various sources, such as:
  - Route values
  - Query strings
  - Headers
  - Request body (for POST/PUT/PATCH)
- These values are then **bound to action parameters or model objects**.
- ASP.NET Core also applies validation attributes (e.g., [Required], [Range]) and records any model-state errors.

4. Filter Pipeline Execution (For MVC and Razor Pages)

ASP.NET Core executes several types of filters before and after the action runs. Filters allow developers to insert cross-cutting logic like logging, authentication, or exception handling in a centralized manner. The filters are executed in the following order:

- **Authorization Filters:** Check user authentication and enforce authorization policies.
- **Resource Filters:** Handle logic related to caching, transactions, or performance timing.
- **Action Filters:** Execute custom logic before and after the controller action runs.
- **Exception Filters:** Catch unhandled exceptions and convert them into meaningful responses.

5. Action or Handler Execution

- After model binding and filters, the **actual application logic** executes.
- For MVC, this means the controller's **action method** runs.
- For Razor Pages, the **page handler** executes.
- For Minimal APIs, the **lambda delegate** or endpoint method runs directly.

This is where the main business logic occurs, such as reading or writing data to a database, processing input, or generating results.

Step 7: Result Generation and Response Preparation

After the handler completes its logic, it produces a **result** that represents the outcome of the request. Examples include:

- **Ok(object)** → Returns 200 OK with JSON data.
- **NotFound()** → Returns 404 Not Found.
- **View()** → Returns an HTML page for Razor views.
- **A Custom Object** → Automatically serialized to JSON or XML depending on content negotiation.

The framework then formats and writes this result into the **HTTP Response**, setting:

- **Status Code:** Indicates the result of the request (e.g., 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error).
- **Headers:** Metadata about the response, such as Content-Type or Cache-Control.
- **Body:** The actual response content (JSON, HTML, XML, or plain text).

Step 8: Response Sent Back to the Client

Finally, the HTTP Response is sent back through the middleware pipeline to the web server (Kestrel or IIS) and then to the client.

What are the Different Types of Routing Supported by ASP.NET Core?

ASP.NET Core supports **two primary routing strategies**, each serving different architectural and organizational needs:

- **Convention-Based Routing:** Configured globally, typically in the Program.cs file.
- **Attribute-Based Routing:** Configured locally using attributes applied directly to controllers and action methods.

Note: Both routing approaches can coexist in the same application. You can use convention-based routing for common, structured endpoints and attribute-based routing for more customized or API-specific routes.

What is Conventional-Based Routing in ASP.NET Core?

Convention-Based Routing defines route patterns in a **centralized location**, usually within the application's startup configuration (e.g., in the Program.cs file). These route patterns act as templates that map parts of the URL to the corresponding **controller** and **action method names**, following a consistent naming convention.

In the **Program.cs** class file, convention-based routing is typically configured using the **MapControllerRoute** method. The Syntax is given below:

```
app.MapControllerRoute(
    name: "default",
    pattern: "api/{controller}/{action}/{id?}");
```

When a request comes in, the **Routing Middleware** examines the URL and matches it against the predefined convention pattern (for example, a pattern like {controller}/{action}/{id?}). Based on this pattern:

- The first part of the URL is treated as the **controller name**.
- The second part identifies the **action method**.
- Any additional segment (like {id?}) is considered a **route parameter**.

This approach offers a predictable, structured way of handling routes, especially in traditional MVC applications where many controllers and actions share similar patterns.

What is Attribute-Based Routing in ASP.NET Core?

Attribute-Based Routing provides developers with full control over route definitions by allowing routes to be declared **directly on controllers and action methods**. Instead of defining all routes globally, you specify them individually using **route attributes** such as [Route], [HttpGet], [HttpPost], [HttpPut], [HttpDelete], etc. The following is the syntax to use Attribute-Based Routing in ASP.NET Core Web API:

```
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id}")]
    public ActionResult GetProduct(int id)
    {
        return Ok($"Product {id}");
    }
}
```

This routing strategy closely aligns with **RESTful API design**, where the structure of the URL and the HTTP verb together represent the resource and the action being performed. It allows you to build expressive, self-documenting, and flexible endpoint structures that can vary from one controller to another.

Example to Understand Routing in ASP.NET Core Web API:

First, create a new ASP.NET Core Web API Application with the name **RoutingInASPNETCoreWebAPI**. Then, add an empty API Controller named EmployeeController within the Controllers folder.

How to Implement Attribute Routing in ASP.NET Core Web API:

Now, let us add two action methods within the EmployeeController. Please don't focus on the return type and the data returned from the action method; instead, concentrate on the Routing concept.

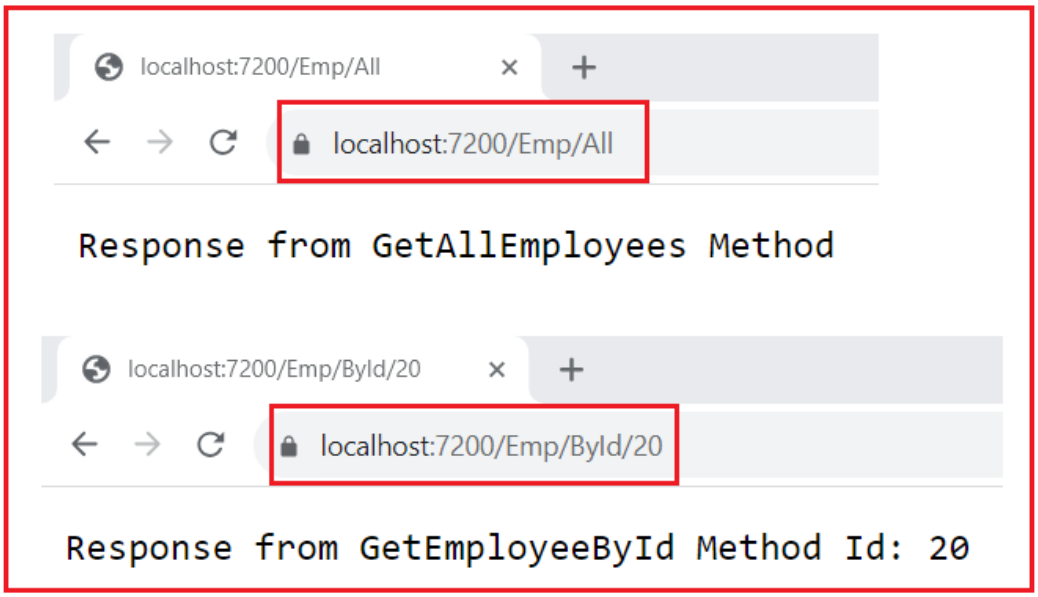
We want to invoke the **GetAllEmployees** method with the URL **/Emp/All** and the **GetEmployeeById** method with the URL **/Emp/ById/102**. To achieve this, we need to use the Route Attribute to decorate the GetAllEmployees and GetEmployeeById methods as **[Route("Emp/All")]** and **[Route("Emp/ById/{Id}")]** respectively. So, modify the EmployeeController class as shown below.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [Route("Emp/All")]
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [Route("Emp/ById/{Id}")]
        [HttpGet]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method Id: {Id}";
        }
    }
}
```

Note: While HTTP verb attributes (e.g., [HttpGet]) are optional, it is recommended to use them for clarity and to ensure proper integration with tools like Swagger. If we don't specify the HTTP verb, such as HttpGet, HttpPost, HttpPut, etc, then by default it will be HttpGet. But if you don't decorate the action method with an HTTP verb, the application will work as expected, but Swagger will not work.

Now, run the application and access the action method using the URL we configured with the Route Attribute. You should get the response as expected, as shown in the image below.



Key Characteristics of Attribute-Based Routing in ASP.NET Core Web API

- **Defined Close to Code:** Routes are defined directly on controller classes and action methods. This makes the routing configuration easier to understand and maintain because it lives alongside the logic it represents.
- **Highly Flexible and Expressive:** Attribute routing allows you to create customized and nested route patterns (for example, /api/customers/{customerId}/orders/{orderId}), which are difficult to achieve using conventional routing.
- **Supports RESTful Design:** Perfect for modern **Web API** development, where routes often depend on resource hierarchies and HTTP verbs (GET, POST, PUT, DELETE). For example, GET /api/products retrieves data, while POST /api/products creates new data, both using the same base path but different HTTP methods.
- **Supports Versioning and Complex Scenarios:** Attribute routing makes it easy to include route parameters, constraints, and even version numbers directly in route templates (e.g., /api/v2/products).
- **Improved Readability:** Since each action's route is explicitly defined, it's easier for developers and API consumers to understand how endpoints map to application logic.
- **Fully Control:** You can easily rename, reorganize, or modify a single route without affecting others. This independence is particularly valuable in large applications with diverse API modules.
- **Commonly Used in ASP.NET Core Web API Applications:** Attribute-based routing is the standard and most preferred routing approach for **ASP.NET Core Web API** projects, where flexibility and RESTful patterns are essential.

How to Implement Conventional-Based Routing in ASP.NET Core Web API:

If you want to use Conventional-Based Routing, then please add the **MapControllerRoute** middleware component as follows in the **Program.cs** class file. Here, we are configuring the Route Pattern as **api/{controller}/{action}/{id?}** where id is the optional parameter.

```
app.MapControllerRoute(
    name: "default",
    pattern: "api/{controller}/{action}/{id?}");
```

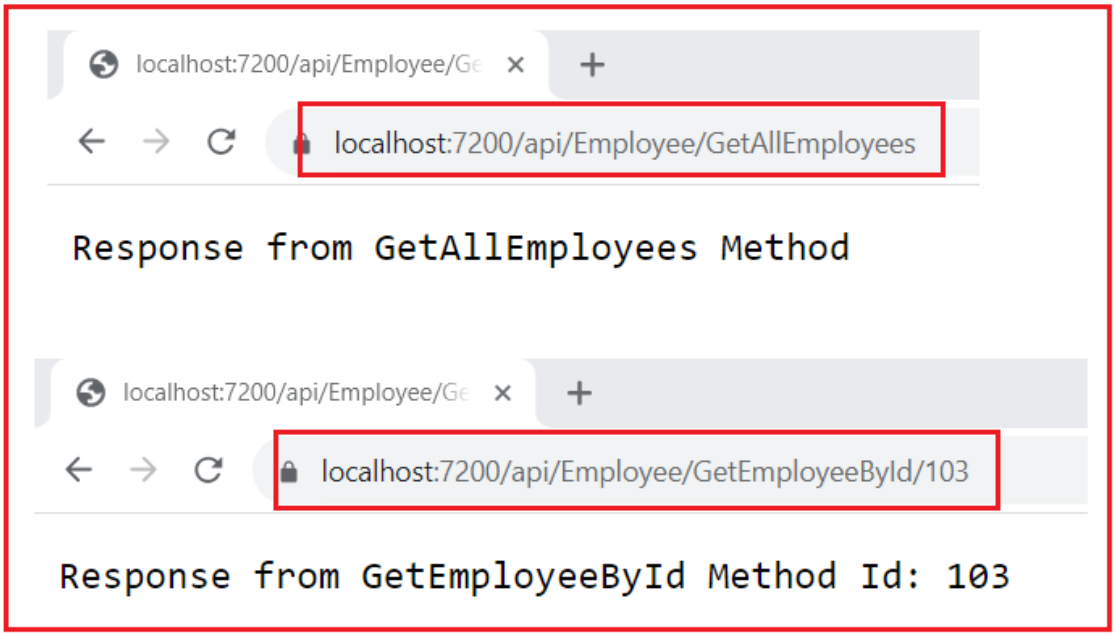
Modifying the Employee Controller:

Next, modify the Employee Controller as follows. Here, we are removing the Route Attribute from both the Controller level and the action method level. We also need to remove the ApiController attribute, which is decorated with the Controller class. This is because the ApiController attribute will force the action method to use the Route attribute.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [HttpGet]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method Id: {Id}";
        }
    }
}
```

With the above changes in place, you can access the APIs using **/api/Employee/GetAllEmployees** and **api/Employee/GetEmployeeById/103** URL as shown in the image below.



Key Characteristics of Convention-Based Routing in ASP.NET Core Web API

- **Centralized Configuration:** All route patterns are defined in one place, usually inside the application startup file (Program.cs). This simplifies route management and provides a single point of control.
- **Consistency Across Controllers:** Since the same pattern applies to multiple controllers and actions, it promotes a uniform and predictable URL structure throughout the application.
- **Ease of Maintenance:** Changing a route convention (for example, adding a prefix or modifying a segment) automatically affects all routes following that pattern, reducing repetitive configuration.
- **Suitable for Structured Applications:** Ideal for larger MVC-based projects where most controllers and actions follow a similar URL structure (for example, /Products/Details/5 or /Orders/Create).
- **Less Flexibility for Complex URLs:** Although it provides uniformity, conventional routing can be restrictive when your application requires highly customized or non-standard URLs, such as RESTful APIs with nested routes or resource-based patterns.
- **Primarily Used in MVC Applications:** Convention-based routing remains the most common choice for **ASP.NET Core MVC web applications**, where routes correspond to controller-action pairs that render HTML views.

Can we use both Conventional-Based and Attribute-Based Routing in ASP.NET Core Web API?

Yes, we can use both Conventional-Based and Attribute-Based Routing in an ASP.NET Core Web API Application. The action method decorated with the Route Attribute will use Attribute Routing, while the action method without the Route Attribute will use Conventional Based Routing. For a better understanding, please modify the Employee Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
```

```
{
    public class EmployeeController : ControllerBase
    {
        [Route("Emp/All")]
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [HttpGet]
        public string GetEmployeeById(int Id)
        {
            return $"Response from GetEmployeeById Method Id: {Id}";
        }
    }
}
```

As you can see in the above code, now we can access the GetAllEmployees method using the Attribute Routing, i.e., **/Emp/All** URL, and we can access the GetEmployeeById action method using the Conventional Routing, i.e., **/api/employee/GetEmployeeById/102** UR,L as shown in the image below:

Using Both Together (Hybrid Routing Approach)

ASP.NET Core allows **combining both routing strategies** within the same application. This hybrid approach is especially useful when:

- Your application contains both **MVC views** (using conventional routing) and **Web API controllers** (using attribute routing).
- You want to keep simple pages following a global pattern, but allow certain controllers or actions to define their own custom routes.

The routing system can evaluate both types of routes under the **Endpoint Routing** system. The order of route evaluation ensures that specific (attribute) routes are typically matched first, followed by general (conventional) ones, maintaining predictable behaviour.

Why can’t we access the same action method in ASP.NET Core Web API using both convention-based and attribute-based routing?

We **cannot access the same action method** using both **convention-based** and **attribute-based routing** because **ASP.NET Core does not combine the two routing systems** for the same controller. When **attribute routing** is detected on a controller (or its actions), **convention-based routing is ignored** for that controller or action. In Web API projects, attribute routing is the **default and recommended approach**.

What Happens Internally

1. During application startup, **NET Core's Endpoint Routing System** discovers all controllers and actions using reflection.
2. For each action, it looks for **attribute routes** such as [Route], [HttpGet], [HttpPost], etc.
3. If an action (or its controller) defines an attribute route, the framework creates a **dedicated endpoint** for that route.
4. Convention-based routes registered via MapControllerRoute() apply **only to actions without any route attributes**.

The discovered endpoints are stored in memory (in **EndpointDataSource**, not a route table as in older ASP.NET MVC). These endpoints are then exposed to the routing middleware via app.UseRouting() and executed via the app.UseEndpoints() or app.MapControllers().

Why This Design Choice?

Mixing both systems for the same action could lead to:

- **Ambiguous Matches**: Multiple endpoints could match the same URL.
- **Maintenance Confusion**: One action might be accessible by several unrelated URLs.
- **Reduced Performance**: Since route resolution would require more checks.

Therefore, ASP.NET Core enforces a clear separation:

- Controllers with **attribute routes** use **endpoint-based attribute routing**
- Controllers without route attributes are handled through **convention-based routes** defined in Program.cs.

Why Swagger is Not Showing Conventional-Based Routes in ASP.NET Core Web API

When you create an ASP.NET Core Web API project and use **Convention-Based Routing**, you might notice that **Swagger (Swashbuckle)** does **not display those endpoints** in the Swagger UI.

At first glance, this may seem like a Swagger configuration issue, but it's actually due to **how ASP.NET Core's routing and Swagger integration work internally**. Let's break it down step by step.

How Swagger Generates the API Documentation?

Swagger (via **Swashbuckle.AspNetCore**) automatically generates API documentation by **scanning for controllers and actions** decorated with routing information. It reads route metadata using **reflection** from attributes such as:

- [Route]
- [HttpGet], [HttpPost], [HttpPut], [HttpDelete]
- [ApiController]

Swagger uses this information to build its **OpenAPI specification (JSON)** and display the endpoints in the Swagger UI. In short, Swagger only documents **Attribute-Based Routes**, because those attributes explicitly define the URL structure that Swagger can extract during startup.

Why Swagger Ignores Conventional-Based Routes?

In **Convention-Based Routing**, routes are defined **centrally** in Program.cs (for example, using MapControllerRoute()), rather than through attributes on the controllers or actions. For example:

```
app.MapControllerRoute(
                name: "default",
                pattern: "api/{controller}/{action}/{id?}");
```

Because the route template is not directly associated with individual controller actions, Swagger **cannot automatically discover or infer** these routes. Swagger expects **endpoint-level metadata** (like [HttpGet] or [Route]) to understand:

- The exact HTTP method (GET, POST, etc.)
- The route template (e.g., /api/products/{id})
- Parameters and their positions in the route

In conventional routing, all of this logic is handled dynamically at runtime by the **Routing Middleware**, not through compile-time attributes. Hence, Swagger's static analysis has no information to display.

By understanding both convention-based and attribute-based routing, as well as how the middleware pipeline processes requests, you can design clean, consistent, and flexible APIs. Attribute routing is typically the recommended approach for modern Web API projects. However, you can mix and match as needed for your particular scenario.

In the next article, I will discuss **Working with Variables and Query Strings in Routing** with Examples. In this article, I explain **Routing in ASP.NET Core Web API Applications** with Examples. I hope you enjoy this article on Routing in ASP.NET Core Web API Applications.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

8 thoughts on “Routing in ASP.NET Core Web API”

HUNEDO
APRIL 20, 2021 AT 9:29 PM

Please update the next tutorial!

Reply

GREG GRAHAM
DECEMBER 30, 2021 AT 2:18 AM

What if you don’t want to use attribute routing?

Reply

DOT NET TUTORIALS
JANUARY 31, 2024 AT 9:34 PM

Then, you need to use Conventional routing.

Reply

MONIKA
JULY 1, 2022 AT 5:47 PM

very good article. It helps me to understand in easy way. thanks.

Reply

ANDREW MCALISTER
JULY 5, 2022 AT 11:29 AM

The project templates must have changed (VS 2022), because it doesn't work.
API not starting

Reply

DOT NET TUTORIALS
JANUARY 31, 2024 AT 9:35 PM

Hi
We have updated the content with .NET 8,

Reply

SAMEERA
FEBRUARY 1, 2025 AT 8:34 PM

Thank you sir.

Reply

MITALI
FEBRUARY 5, 2025 AT 1:20 PM

Superb explanation.

Reply